



Lab 03

COMP1322 Programming II

Student Name: Qi Liu

Student ID: 36557854

Student Email: ql1e24@soton.ac.uk

Date: March 5, 2026

1: Java Basics Recap

1.1. Class & Object Creation

Code:

```
1  class Person { 4个用法
2
3      String name; // stores the name 2个用法
4      int age; // stores the age 2个用法
5
6      Person(String name, int age) { 2个用法
7          this.name = name; // "this.name" means the class variable, "name" is what we passed in
8          this.age = age;
9      }
10
11     // Printing person's info
12     void displayInfo() { 2个用法
13         System.out.println("Name: " + name);
14         System.out.println("Age: " + age);
15     }
16 }
17
18 public class Task1_1_Person {
19     public static void main(String[] args) {
20         Person person1 = new Person(name: "Max", age: 20); // create the first Person object
21
22         Person person2 = new Person(name: "Lando", age: 25); // create the second one
23
24         person1.displayInfo(); // call displayInfo() on each person to print their details
25         System.out.println("---");
26         person2.displayInfo();
27     }
28 }
```

Output:

```
Name: Max
Age: 20
---
Name: Lando
Age: 25
```

Describe what an object is and how it is created in Java.

An object is created from a class, which contains attributes and methods. And attributes work like variables which are used for storing data. Then method is kind of like a function, it can define actions that an object can perform.

And the method to create an object is to declare a reference variable at the beginning, then create the object using **new** keyword. In the end we assign it to the reference so that we create an object successfully.

1.2. Variable Types & Scope

Code:

```
1  import java.util.ArrayList; // ArrayList
2
3  public class Task1_2_PassByValue {
4      static void tryToChangeNumber(int num) { 1个用法
5          num = 999; // change the copy, not the original
6          System.out.println("Inside method, num = " + num);
7      }
8
9      static void addToList(ArrayList<Integer> list) { 1个用法
10         list.add(999); // adds 999 to the actual list
11         System.out.println("Inside method, list = " + list);
12     }
13
14     public static void main(String[] args) {
15         int myNumber = 5; // create an int number
16         System.out.println("Before method call, my number is: " + myNumber);
17         tryToChangeNumber(myNumber); // try to change it
18         System.out.println("After method call, my number is: " + myNumber);
19
20         System.out.println("---");
21
22         // Pass by Reference example
23         ArrayList<Integer> myList = new ArrayList<>(); // create a list
24         // add numbers
25         myList.add(1);
26         myList.add(2);
27         myList.add(3);
28         System.out.println("Before method call, my list is: " + myList); // [1, 2, 3]
29         addToList(myList); // add 999
30         System.out.println("After method call, my list is: " + myList); // [1, 2, 3, 999]
31     }
32 }
```

Output:

```
Before method call, my number is: 5
Inside method, num = 999
After method call, my number is: 5
---
Before method call, my list is: [1, 2, 3]
Inside method, list = [1, 2, 3, 999]
After method call, my list is: [1, 2, 3, 999]
```

Explain the difference between pass-by-value and pass-by-reference with examples.
pass-by-value is created for copying the variable's value and then passing it to the method that we are calling next. And whatever any kind of changes inside the method will not affect the original variable.

2: Methods & Overloading

2.1. Method Overloading Practice

Code:

```
1 public class Task2_1_Calculator {  
2  
3     int add(int a, int b) { 1个用法  
4         return a + b; // just add them and return the result  
5     }  
6  
7     double add(double a, double b) { 1个用法  
8         return a + b; // add them and return the result  
9     }  
10  
11     String add(String a, String b) { 1个用法  
12         return a + b; // joins strings together  
13     }  
14  
15     public static void main(String[] args) {  
16         Task2_1_Calculator calc = new Task2_1_Calculator(); // create a Calculator object  
17  
18         int result1 = calc.add(a: 6, b: 7); // call the int version  
19         System.out.println("int add: " + result1);  
20  
21         double result2 = calc.add(a: 6.6, b: 7.7); // call the double version  
22         System.out.println("double add: " + result2);  
23  
24         String result3 = calc.add(a: "Hello!", b: "I am a Year 1 student at the University of Southampton"); // call the String version  
25         System.out.println("String add: " + result3);  
26     }  
27 }
```

Output:

```
int add: 13  
double add: 14.3  
String add: Hello! I am a Year 1 student at the University of Southampton
```

What is method overloading? Why is it useful?

Method overloading means defining multiple methods with the same name in the same class. But they are under different parameter lists.

Why it is useful is that it uses the same method name for similar operations so that we only need to remember one method name. And the compiler determines which method to execute during compilation.

2.2. Calling Static & Non-Static Methods

Code:

```

1  public class Task2_2_Greeting {
2      static void sayHello() { 1个用法
3          System.out.println("Hello, World!"); // static method can call it without creating an object
4      }
5
6      void personalizedGreeting(String name) { 2个用法
7          System.out.println("Hello, " + name + "!"); // non-static method need to create an object first
8      }
9
10     public static void main(String[] args) {
11         Task2_2_Greeting.sayHello(); // call the static method
12
13         Task2_2_Greeting greet = new Task2_2_Greeting(); // create the object
14         greet.personalizedGreeting(name: "Alice"); // now we can call the method
15         greet.personalizedGreeting(name: "Bob"); // call it again with a different name
16     }
17 }

```

Explain the difference between static and non-static methods with examples.

A static method belongs to the class instead of an object. It can be called without creating an object and shared by all objects of the class.

Non-static methods belong to object. And before we use it we have to create an object first and every object can use the method.

3: Encapsulation & Java Library Usage

3.1. Encapsulation

Code:

```

1 public class Task3_1_BankAccount {
2     private String accountNumber; // stores account number 2个用法
3     private double balance; // stores money amount 5个用法
4
5     Task3_1_BankAccount(String accountNumber, double startingBalance) { 1个用法
6         this.accountNumber = accountNumber; // set account number
7         this.balance = startingBalance; // set starting balance
8     }
9
10    double getBalance() { 1个用法
11        return balance; // return current balance
12    }
13
14    void deposit(double amount) { 1个用法
15        if (amount > 0) { // check the amount is positive or not
16            balance = balance + amount; // add to balance
17            System.out.println("Deposited: " + amount);
18        } else {
19            System.out.println("Invalid deposit amount!"); // reject negative input
20        }
21    }
22
23    // shows the account info
24    void displayInfo() { 1个用法
25        System.out.println("Account Number: " + accountNumber); // print account number
26        System.out.println("Balance: " + balance); // print balance
27    }
28
29    public static void main(String[] args) {
30        // Create a bank account object
31        Task3_1_BankAccount account = new Task3_1_BankAccount("Account-001", 667.7);
32
33        account.displayInfo(); // show starting info
34
35        account.deposit(7766.0); // deposit some money
36
37        System.out.println("New balance: " + account.getBalance()); // check balance using getter
38    }
39 }

```

Output:

```

Account Number: Account-001
Balance: 667.7
Deposited: 7766.0
New balance: 8433.7

```

Why is encapsulation important? How does it improve security in programming?

Encapsulation can make sure that data won't be changed directly. In a real developing environment, variables are usually made private, and methods like getters and setters are used to access them.

Since we can not change the data directly, other classes cannot change the variable directly. And methods can check if the data is valid before program saving it.

3.2. Using Java Collections

Code:

```
1  import java.util.ArrayList;
2  import java.util.Iterator;
3
4  public class Task3_2_StudentList {
5
6      public static void main(String[] args) {
7          ArrayList<String> students = new ArrayList<>(); // create an ArrayList to store student names
8
9          // add student names to the list
10         students.add("Max");
11         students.add("Lando");
12         students.add("Charlies");
13         students.add("Lewis");
14
15         System.out.println("Student Names:");
16
17         Iterator<String> iterator = students.iterator(); // get the iterator one by one to go through the list
18
19         // checks if there is another item left in the list
20         while (iterator.hasNext()) {
21             String name = iterator.next(); // next() gives us the next item
22             System.out.println(name);
23         }
24     }
25 }
```

Output:

```
Student Names:
Max
Lando
Charlies
Lewis
```

What is an Iterator, and why is it useful for collections in Java?

An Iterator allows me to loop through elements in a collection at a time but we need to import `java.util.Iterator` first.

It is useful because it is easy to traverse collections and allows elements to be removed during iteration.

4: Inheritance & Polymorphism

4.1. Implementing Inheritance

Code:

```
1  import java.util.ArrayList;
2
3  // superclass (parent class)
4  class Animal { 4 个用法 2 个继承者
5      String name; 4 个用法
6
7      // constructor for Animal
8      Animal(String name) { 2 个用法
9          |   this.name = name; // set the name
10         |   }
11
12         // default sound method
13         void makeSound() { 1 个用法 2 个重写
14             |   System.out.println(name + "Animal makes a sound"); // default message
15             |   }
16     }
17
18     // subclass of animal (child class) dog
19     class Dog extends Animal { 2 个用法
20         Dog(String name) { // constructor for dog 2 个用法
21             |   super(name); // calls Animal's constructor to set the name
22             |   }
23
24         @Override // replacing the parent's makeSound() method 1 个用法
25         void makeSound() {
26             |   System.out.println(name + ": Bark!");
27             |   }
28     }
29
30     // subclass of Animal cat
31     class Cat extends Animal { 2 个用法
32         Cat(String name) { // constructor for cat 2 个用法
33             |   super(name); // call parent constructor
34             |   }
35
36         @Override 1 个用法
37         void makeSound() {
38             |   System.out.println(name + ": Meow!");
39             |   }
40     }
```

```

42 ▶ public class Task4_1_Animals {
43
44 ▶     public static void main(String[] args) {
45         ArrayList<Animal> animals = new ArrayList<>(); // holds Animal objects
46
47         animals.add(new Dog( name: "Ab"));
48         animals.add(new Cat( name: "Bob"));
49         animals.add(new Dog( name: "Buddy"));
50         animals.add(new Cat( name: "Charlie"));
51
52         for (Animal a : animals) {
53             a.makeSound(); // calls Dog or Cat version depending on actual type
54         }
55     }
56 }

```

Output:

```

Ab: Bark!
Bob: Meow!
Buddy: Bark!
Charlie: Meow!

```

Why does IntelliJ prompt you to add `@Override`?

Because the annotation `@Override` is a method that overrides a method in the superclass or interface. And it can help programs to detect potential errors and make the code easier to read.

What happens if you don't add it? Will the program throw an error?

Even though we don't add it, the code still can run successfully, but it may not detect other possible errors, such as the wrong method names.

What is the actual purpose of `@Override` in Java?

The actual purpose is used for error checking and code/method clarity.

4.2. Method Overriding & Superclass Reference

Code:

```
1 class Animal2 { 2个用法 1个继承者
2     String name; 4个用法
3
4     Animal2(String name) { 1个用法
5         this.name = name;
6     }
7
8     void makeSound() { 1个用法 1个重写
9         System.out.println(name + ": Animal makes a sound");
10    }
11
12    // added to superclass
13    void sleep() { 1个用法
14        System.out.println(name + " is sleeping... Zzz");
15    }
16 }
17
18 class Dog2 extends Animal2 { 1个用法
19     Dog2(String name) { 1个用法
20         super(name); // call parent constructor
21     }
22
23     @Override 1个用法
24     void makeSound() {
25         System.out.println(name + ": Bark!");
26     }
27 }
28
29 public class Task4_2_DynamicDispatch {
30
31     public static void main(String[] args) {
32         Animal2 a = new Dog2( name: "Max"); // create a Dog2 object but reference it as an Animal2
33
34         a.makeSound(); // calls Dog2
35         a.sleep();     // calls Animal2's sleep() because Dog2 doesnt override it
36     }
37 }
```

Output:

```
Max: Bark!
Max is sleeping... Zzz
```

How does dynamic method dispatch work in Java?

It occurs when a reference to the superclass points to an object of the subclass that has overridden the method.

5: Advanced Java Library - HashMaps

5.1. Storing Key-Value Pairs

Code:

```
1 import java.util.HashMap; // HashMap
2
3 public class Task5_1_HashMap {
4
5     public static void main(String[] args) {
6         HashMap<String, Integer> ages = new HashMap<>(); // create a HashMap to stores pairs of (key, value)
7
8         // Add key and value
9         ages.put("Max", 20);
10        ages.put("Lando", 25);
11        ages.put("Charlies", 22);
12
13        // Retrieve a value using the key
14        int maxAge = ages.get("Max"); // get Max's age
15        System.out.println("Max's age: " + maxAge);
16
17        System.out.println("All entries:");
18        for (String name : ages.keySet()) { // gives us all keys
19            System.out.println(name + " is " + ages.get(name));
20        }
21    }
22 }
```

Output:

```
Max's age: 20
All entries:
Max is 20
Charlies is 22
Lando is 25
```

How does a HashMap work in Java? What are its advantages?

HashMap can store data in pairs of keys and values and uses a hash function to determine the bucket for each key.

The advantages of a HashMap:

Keys are unique, and values can be duplicated.

Null key (only one) and multiple null values are allowed.

Uses hashing for fast access.

Linked lists are used for avoiding collisions.

5.2. Using Generics Properly

Code:

```
1 import java.util.ArrayList; // need ArrayList
2
3 public class Task5_2_Generics {
4
5     public static void main(String[] args) {
6         ArrayList listWithoutGenerics = new ArrayList(); // no type specified
7         listWithoutGenerics.add("Hello"); // add a String
8         listWithoutGenerics.add(123); // add an Integer
9         listWithoutGenerics.add(3.14); // add a Double
10
11         System.out.println("Without generics list: " + listWithoutGenerics);
12
13         try {
14             String s = (String) listWithoutGenerics.get(1); // get(1) is 123 (an Integer) but I try to cast it to String
15         } catch (ClassCastException e) {
16             System.out.println("Runtime error: " + e.getMessage()); // catch the crash
17         }
18
19         System.out.println("---");
20
21         ArrayList<String> listWithGenerics = new ArrayList<>(); // only Strings allowed
22         listWithGenerics.add("Hello");
23         listWithGenerics.add("World");
24
25         System.out.println("With generics list: " + listWithGenerics);
26
27         // No casting needed due to it is always a String
28         String s = listWithGenerics.get(0);
29         System.out.println("First item: " + s);
30     }
31 }
```

Output:

```
Without generics list: [Hello, 123, 3.14]
Runtime error: class java.lang.Integer cannot be cast to class java.lang.String (java.lang.Integer and java.lang.String are in module java.base of loader 'bootstrap')
---
With generics list: [Hello, World]
First item: Hello
```

What are Generics, and why should we use them in Java collections?

In a collection, generics avoid the addition of wrong types of objects, eliminating the need for casting.

2: Library Management System

Code:

```

1  import java.util.ArrayList;
2  import java.util.Scanner;
3
4  // superclass Publication for all types of publications
5  class Publication { 4 个用法 2 个继承者
6
7      // private makes only this class can access them
8      private String title; 3 个用法
9      private String author; 3 个用法
10     private int publicationYear; 3 个用法
11
12     // constructor to set up the publication
13     Publication(String title, String author, int publicationYear) { 2 个用法
14         this.title = title;
15         this.author = author;
16         this.publicationYear = publicationYear;
17     }
18
19     // get title
20     String getTitle() { 0 个用法
21         return title;
22     }
23
24     // get author
25     String getAuthor() { 0 个用法
26         return author;
27     }
28
29     // get publicationYear
30     int getPublicationYear() { 0 个用法
31         return publicationYear;
32     }
33
34     // display publication info
35     void displayInfo() { 3 个用法 2 个重写
36         System.out.println("Title: " + title);
37         System.out.println("Author: " + author);
38         System.out.println("Publication Year: " + publicationYear);
39     }
40 }
41

```

```

42 // child class of publication - Book
43 class Book extends Publication { 2个用法
44
45     private String ISBN; // unique ID for the book 3个用法
46
47     // constructor
48     Book(String title, String author, int publicationYear, String ISBN) { 1个用法
49         super(title, author, publicationYear); // call publication constructor
50         this.ISBN = ISBN; // set the ISBN
51     }
52
53     // get ISBN
54     String getISBN() { 0个用法
55         return ISBN;
56     }
57
58     @Override // Override displayInfo() to also show the ISBN 3个用法
59     void displayInfo() {
60         super.displayInfo(); // call the parent method to print title, author, year
61         System.out.println("ISBN: " + ISBN); // add ISBN on top
62     }
63 }
64
65 class Magazine extends Publication { 2个用法
66
67     private int issueNumber; // issue of magazine 3个用法
68
69     Magazine(String title, String author, int publicationYear, int issueNumber) { 1个用法
70         super(title, author, publicationYear); // call publication constructor
71         this.issueNumber = issueNumber; // set the issue number
72     }
73
74     // get issueNumber
75     int getIssueNumber() { 0个用法
76         return issueNumber;
77     }
78
79     @Override // show the issue number 3个用法
80     void displayInfo() {
81         super.displayInfo(); // print title, author, year
82         System.out.println("Issue Number: " + issueNumber); // add issue number
83     }
84 }

```

```

86 public class LibraryManagementSystem {
87     public static void main(String[] args) {
88         Scanner scanner = new Scanner(System.in);
89         ArrayList<Publication> library = new ArrayList<>(); // store all publications (books and magazines)
90
91         System.out.println("Enter the number of books to add:");
92         int numBooks = Integer.parseInt(scanner.nextLine()); // read number of books
93
94         // loop to get details for each book
95         for (int i = 1; i <= numBooks; i++) {
96             System.out.println("Enter Book " + i + " details:");
97
98             System.out.print("Title: ");
99             String title = scanner.nextLine();
100
101             System.out.print("Author: ");
102             String author = scanner.nextLine();
103
104             System.out.print("Publication Year: ");
105             int year = Integer.parseInt(scanner.nextLine());
106
107             System.out.print("ISBN: ");
108             String isbn = scanner.nextLine();
109
110             Book book = new Book(title, author, year, isbn); // create a new Book object with the input info
111
112             library.add(book); // add the book to list
113         }
114
115         // get Magazines from user
116         System.out.println("Enter the number of magazines to add:");
117         int numMagazines = Integer.parseInt(scanner.nextLine()); // read number of magazines
118
119         // loop to get details for each magazine
120         for (int i = 1; i <= numMagazines; i++) {
121             System.out.println("Enter Magazine " + i + " details:"); // tell user which magazine
122
123             System.out.print("Title: ");
124             String title = scanner.nextLine(); // read title
125
126             System.out.print("Author: ");
127             String author = scanner.nextLine(); // read author

```



```

128
129         System.out.print("Publication Year: ");
130         int year = Integer.parseInt(scanner.nextLine()); // read year
131
132         System.out.print("Issue Number: ");
133         int issueNum = Integer.parseInt(scanner.nextLine()); // read issue number
134
135         // Create a Magazine object with the info we got
136         Magazine magazine = new Magazine(title, author, year, issueNum);
137
138         library.add(magazine); // add the magazine to our library list
139     }
140
141     System.out.println("\nLibrary Collection:");
142
143     // loop through every item in the library ArrayList
144     for (Publication pub : library) {
145         pub.displayInfo();
146         System.out.println("---");
147     }
148
149     scanner.close();
150 }
151 }

```

Output:

Enter the number of books to add:

2

Enter Book 1 details:

Title: *Harry Potter*

Author: *JK*

Publication Year: *1996*

ISBN: *123213*

Enter Book 2 details:

Title: *67*

Author: *Skim*

Publication Year: *2005*

ISBN: *23414*

Enter the number of magazines to add:

1

Enter Magazine 1 details:

Title: *wdsfsdf*

Author: *XIii*

Publication Year: *2025*

Issue Number: *12323*

Library Collection:

Title: Harry Potter

Author: JK

Publication Year: 1996

ISBN: 123213